

เครื่องยิงลูกบอล

มาตินาได้ประดิษฐ์เครื่องยิงลูกบอล (Ball Machine) เพื่อสร้างความบันเทิงให้แก่ผู้เข้าร่วมการแข่งขัน IOI โดยโครงสร้างภายในของเครื่องยิงลูกบอลมีดังต่อไปนี้

- เครื่องยิงลูกบอลนี้ประกอบด้วย **ปม (node)** N ปม แต่ละปมระบุด้วยหมายเลขตั้งแต่ 0 ถึง $N - 1$ โดยปม $N - 1$ เรียกว่า **ปมราก**
- สำหรับแต่ละปม u ที่มี $0 \leq u < N - 1$ **ปมพ่อ**ของปม u คือ ปม $P[u]$ ที่มี $P[u] > u$ และปม u เป็น**ปมลูก**ของ $P[u]$ โดยปมรากนั้นไม่มีปมพ่อ และให้ปมที่ไม่มีปมลูกเรียกว่า **ปมใบ**

คุณไม่ทราบค่า N หรือปมพ่อ $P[u]$ ของปม u ใด ๆ แต่มาตินาจะบอกคุณเพียง M ซึ่งเป็นจำนวนของปมใบ และหมายเลขของปมใบเหล่านั้นคือ $0, 1, \dots, M - 1$ งานของคุณคือ การตรวจสอบโครงสร้างภายในของเครื่องยิงลูกบอล โดยการใช้งานเครื่องยิงลูกบอลนั้น

ปมแต่ละปมของเครื่องสามารถบรรจุลูกบอลได้มากที่สุด**หนึ่งลูก** และแต่ละลูกมี**ค่า**ซึ่งเป็นจำนวนเต็มที่ไม่เป็นลบ เรากล่าวว่าปมมีค่า x ถ้าปมนั้นบรรจุลูกบอลที่มีค่า x ปมจะถือว่า**ถูกรอบครอง**หากมีลูกบอลบรรจุอยู่ภายใน มิฉะนั้น ปมนั้นจะถือว่า**ว่าง** โดยในตอนเริ่มต้นปมทุกปมจะว่างอยู่

คุณสามารถดำเนินการได้สองประเภท ดังนี้

- insert(U, X)**: พยายามแทรกลูกบอลที่มีค่า X ลงในปมใบ U
 - หากปมใบ U ถูกรอบครอง การดำเนินการจะส่งคืนค่า `false` โดยไม่ใส่ลูกบอลเข้าไป
 - หากปมใบ U ว่างอยู่ ลูกบอลจะถูกวางไว้ที่ปม U จากนั้นลูกบอลจะเคลื่อนที่ไปยังปมพ่อของปมที่ลูกบอลอยู่ซ้ำไปเรื่อย ๆ トラバタがパムフンワナワナ 移動する時、ลูกบอลไปถึงปมรากหรือถึงปมที่มีปมพ่อถูกรอบครองอยู่ การดำเนินการนี้จะคืนค่า `true`
- collect()**: ถ้าปมรากว่าง (ซึ่งหมายความว่าเครื่องยิงลูกบอลไม่มีลูกบอลอยู่เลย) `collect` จะคืนค่าเป็นอาร์เรย์ว่าง มิฉะนั้น ฟังก์ชัน `traverse` ซึ่งเป็นขั้นตอนแบบเรียกซ้ำ ตามที่อธิบายไว้ด้านล่างนี้ จะรวบรวมค่าของลูกบอลทั้งหมดที่อยู่ในเครื่องในขณะนั้นลงในอาร์เรย์ S โดยในตอนเริ่มต้นอาร์เรย์ S นั้นว่างเปล่า และเริ่มด้วยการเรียกใช้ `traverse( $N - 1$ )`

`traverse(u)` :

นำค่าของลูกบอลที่ปม u ไปต่อท้าย S

ให้ $c[u]$ เป็นลำดับของปมลูกของ u

เรียงลำดับ $c[u]$ จากน้อยไปมากตามค่าของลูกบอลที่ปมเหล่านั้น

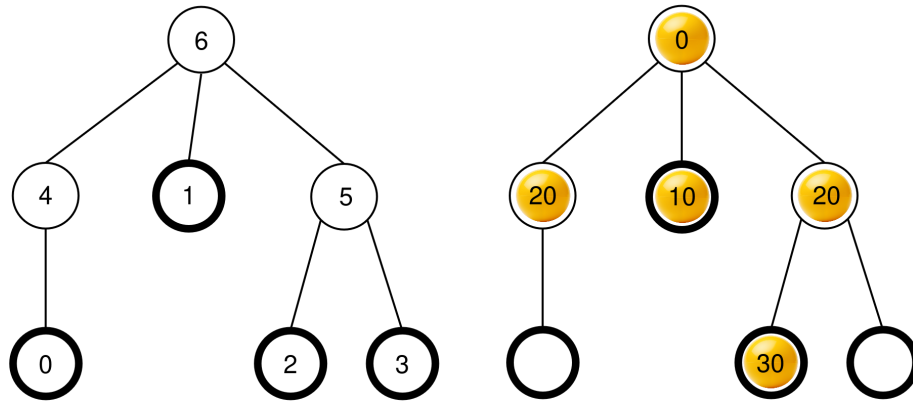
(หากมีหลายปมที่มีค่าเท่ากัน ปมเหล่านั้นสามารถเรียงลำดับใดก็ได้)

สำหรับแต่ละ v ใน $c[u]$:

`traverse(v)`

หลังจากขั้นตอนนี้เสร็จสิ้น ลูกบอลทั้งหมดจะถูกนำออก จากเครื่อง และ `collect` จะคืนค่า S

ตัวอย่างเช่น พิจารณาเครื่องที่มี $N = 7$ ปม และอาร์เรย์ปมพ่อ $P = [4, 6, 5, 5, 6, 6]$ ภาพด้านซ้ายแสดงเครื่องยัง ลูกบอลที่ว่างเปล่าพร้อมหมายเลขปม และภาพด้านขวาแสดงการจัดเรียงลูกบอลที่เป็นไปได้หลังจากลำดับการดำเนินการ `insert` บางอย่าง (ลำดับการเรียก `insert` นี้แสดงอยู่ในหัวข้อตัวอย่าง)



เมื่อมีการเรียก `collect()` ปมราก (ปม 6) ถูกครอบครองอยู่ ดังนั้น S จะถูกกำหนดค่าเริ่มต้นเป็น $S = []$ แล้วจึงเรียก `traverse(6)`

`traverse(6)`: ปม 6 มีค่า 0 และถูกนำไปต่อท้าย S จะได้ $S = [0]$ ปมลูกของ 6 คือ 4, 1 และ 5 ซึ่งทั้งหมดถูกครอบครองอยู่แล้ว โดยมีค่าเป็น 20, 10 และ 20 ตามลำดับ การเรียงลำดับจากน้อยไปมากจะได้ $c[6] = [1, 5, 4]$ (โปรดทราบว่า $c[6] = [1, 4, 5]$ ก็ใช้ได้เช่นกัน เนื่องจากปม 4 และ 5 มีค่าเท่ากัน) จากนั้นฟังก์ชันจะเรียก `traverse` บนแต่ละปมใน $c[6]$ ตามลำดับนั้น

- **`traverse(1)`**: ปม 1 มีค่า 10 เมื่อนำไปต่อท้าย S จะทำให้ $S = [0, 10]$ ปม 1 ไม่มีปมลูกที่ถูกครอบครอง ดังนั้นการเรียกนี้จึงสิ้นสุดลง
- **`traverse(5)`**: ปม 5 มีค่า 20 เมื่อนำไปต่อท้าย S จะได้ $S = [0, 10, 20]$ ปม 5 มีปมลูกที่ถูกครอบครองอยู่หนึ่งปม คือ ปม 2 ดังนั้น $c[5] = [2]$ และขั้นตอนจะเรียก `traverse(2)`
 - **`traverse(2)`**: ปม 2 มีค่า 30 เมื่อนำไปต่อท้าย S จะได้ $S = [0, 10, 20, 30]$ ปม 2 ไม่มีปมลูกที่ถูกครอบครอง ดังนั้นการเรียกนี้จึงสิ้นสุดลง
 - การดำเนินการจะกลับไปยัง `traverse(5)` ซึ่งได้ประมวลผลลูกทั้งหมดใน $c[5]$ ดังนั้นการเรียกนี้จึงสิ้นสุดลง
- **`traverse(4)`**: ปม 4 มีค่า 20 เมื่อนำไปต่อท้าย S จะได้ $S = [0, 10, 20, 30, 20]$ ปม 4 ไม่มีปมลูกที่ถูกครอบครอง ดังนั้นการเรียกนี้จึงสิ้นสุดลง

การเรียกทั้งหมดเสร็จสมบูรณ์แล้ว และไม่มีปมใดให้ประมวลผลเพิ่มเติมอีก ในขั้นตอนสุดท้าย ลูกบอลทุกลูกจะถูกนำออก จากเครื่องยังลูกบอล และ `collect` จะคืนอาร์เรย์ $S = [0, 10, 20, 30, 20]$

งานของคุณคือ การคำนวณจำนวนปม N และแสดงอาร์เรย์ของปมพ่อ $R = [R[0], R[1], \dots, R[N-2]]$ ที่อธิบาย โครงสร้างของเครื่องยังลูกบอล โดยที่การระบุหมายเลขของปมที่ไม่ใช่ปมใบและไม่ใช่ปมรากอาจแตกต่างกันได้ ตามหลัก การแล้ว อาร์เรย์ที่แสดง R จะถือว่าถูกต้องก็ต่อเมื่อสามารถกำหนดหมายเลขของปมที่ไม่ซ้ำกัน $L[u]$ โดยที่ $0 \leq L[u] < N$ ให้กับแต่ละปม u ของเครื่องได้ โดยมีเงื่อนไขดังนี้

- $L[u] = u$ สำหรับทุก $0 \leq u < M$ และ $u = N - 1$, และ
- $L[P[u]] = R[L[u]]$ สำหรับทุก $0 \leq u < N - 1$

ไม่จำเป็นว่า $R[u]$

มันไม่จำเป็นว่า $R[u] > u$ และเครื่องยิงลูกบอลจะต้องไม่มีลูกบอลอยู่เมื่อคุณคืนคำตอบคำตอบของคุณ

ให้ K เป็นจำนวนการดำเนินการ `collect` ที่เกิดขึ้น และ B เป็นค่าสูงสุดของลูกบอลใด ๆ จากการเรียกใช้ `insert` ทั้งหมด กำหนดให้ $C = K + B$ และ C **ต้องไม่เกิน** 1000 ซึ่งคะแนนของคุณในบางปัญหาย่อยขึ้นอยู่กับค่าของ C

รายละเอียดการเขียนโปรแกรม

คุณจะต้องเขียนฟังก์ชันต่อไปนี้

```
std::vector<int> find_structure(int M)
```

- M : จำนวนปมในเครื่องยิงลูกบอล
- ฟังก์ชันนี้จะถูกเรียกหนึ่งครั้งพอดีสำหรับแต่ละข้อมูลทดสอบ
- ฟังก์ชันนี้จะต้องคืนอาร์เรย์ $R = [R[0], R[1], \dots, R[N - 2]]$ ที่มีขนาด $N - 1$ ซึ่งคืออาร์เรย์ของปมพ้อที่ถูกต้อง

ฟังก์ชันของคุณจะต้องเรียกสองฟังก์ชันต่อไปนี้เพื่อทำการใด ๆ กับเครื่องยิงลูกบอล

ฟังก์ชันแรกคือ

```
bool insert(int U, int X)
```

- U : หมายเลขของปมในที่จะใส่ลูกบอล โดยมีเงื่อนไขคือ $0 \leq U < M$
- X : ค่าจำนวนเต็มของลูกบอล โดยมีเงื่อนไขคือ $0 \leq X \leq 1000$
- ฟังก์ชันนี้จะคืนค่า `true` เมื่อการใส่ลูกบอลสำเร็จ และจะคืนค่า `false` เมื่อปมใน U นั้นไม่ว่าง
- ฟังก์ชันนี้ถูกเรียกได้ไม่เกิน 500 000 ครั้ง

ฟังก์ชันที่สองคือ

```
std::vector<int> collect()
```

- รวบรวมค่าของลูกบอลที่ถูกใส่ไป หลังจากนั้นจึงทำให้เครื่องยิงลูกบอลว่าง แล้วคืนค่าอาร์เรย์ S ของการเก็บลูกบอล

หากขณะใดขณะหนึ่งระหว่างการทำงานของโปรแกรมของคุณมีค่าของ C เกิน 1000 โปรแกรมของคุณจะได้รับผลการตรวจเป็น `Output isn't correct: Too many resources used`

โครงสร้างของเครื่องยิงลูกบอลจะถูกกำหนดไว้ตายตัวก่อนที่ `find_structure` จะถูกเรียก การทำงานของเกรดเดอร์นั้น **Deterministic** กล่าวคือหากคุณทดลองใช้เกรดเดอร์สองครั้งและทำการเรียกฟังก์ชันต่าง ๆ ในลำดับที่เหมือนกัน

ผลการเรียก collect จะเหมือนกันเสมอ

ข้อกำหนด

- $2 \leq N \leq 1000$
- $1 \leq M \leq 200$
- $M < N$

ปัญหาย่อย

ปัญหาย่อย	คะแนน	เงื่อนไขเพิ่มเติม
1	5	ปมรากมีปมลูกจำนวน M ปมเสมอ
2	10	$M \leq 3$
3	25	$N \leq 200, M \leq 45$
4	60	ไม่มีเงื่อนไขเพิ่มเติม

ในปัญหาย่อย 3 และ 4 คะแนนของคุณจะขึ้นอยู่กับค่า C ดังนี้

ปัญหาย่อย 3 (25 คะแนน)

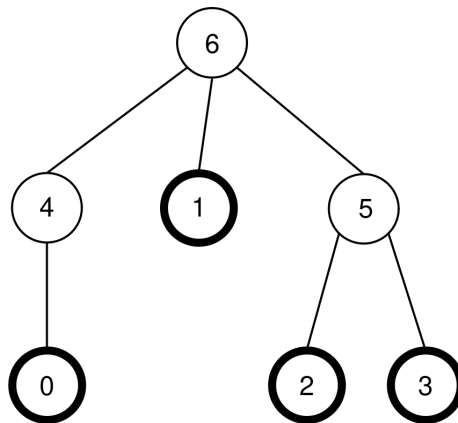
ข้อกำหนด	คะแนน
$1000 < C$	0
$45 < C \leq 1000$	13
$C \leq 45$	25

ปัญหาย่อย 4 (60 คะแนน)

ข้อกำหนด	คะแนน
$1000 < C$	0
$200 < C \leq 1000$	7
$71 < C \leq 200$	$47 - \frac{C}{5}$
$44 < C \leq 71$	$104 - C$
$C \leq 44$	60

ตัวอย่าง

ให้พิจารณาเครื่องยิงลูกบอลในโจทย์ข้างต้น ซึ่งมี $N = 7$, $M = 4$, และ $P = [4, 6, 5, 5, 6, 6]$ ดังแสดงอีกครั้งในรูปต่อไปนี้



เกร็ดเดอร์จะเรียกฟังก์ชันต่อไปนี้

```
find_structure(4)
```

ฟังก์ชันนี้สามารถเรียกฟังก์ชันต่าง ๆ ดังนี้

1. **insert(0, 0):** ปมใบ 0 ว่าง ดังนั้นลูกบอลที่มีค่า 0 จึงถูกวางที่ปมใบ 0 และเนื่องจากปม $P[0] = 4$ ว่าง ลูกบอลจึงเคลื่อนไปยังปม 4 และเนื่องจากปม $P[4] = 6$ ว่าง ลูกบอลจึงเคลื่อนไปยังปม 6 และเนื่องจากปม 6 คือ ปมราก ลูกบอลจึงหยุดเคลื่อนที่ ฟังก์ชันนี้คืนค่า `true`
2. **insert(3, 20):** ปมใบ 3 ว่าง ดังนั้นลูกบอลที่มีค่า 20 จึงถูกวางที่ปมใบ 3 และเนื่องจากปม $P[3] = 5$ ว่าง ลูกบอลจึงเคลื่อนไปยังปม 5 และเนื่องจากปม $P[5] = 6$ ไม่ว่าง ลูกบอลจึงหยุดเคลื่อนที่ ฟังก์ชันนี้คืนค่า `true`
3. **insert(1, 10):** ปมใบ 1 ว่าง ดังนั้นลูกบอลที่มีค่า 10 จึงถูกวางที่ปมใบ 1 และเนื่องจากปม $P[1] = 6$ ไม่ว่าง ดังนั้นลูกบอลจึงยังคงอยู่ที่ปม 1 ฟังก์ชันนี้คืนค่า `true`
4. **insert(2, 30):** ปมใบ 2 ว่าง ดังนั้นลูกบอลที่มีค่า 30 จึงถูกวางที่ปมใบ 2 และเนื่องจากปม $P[2] = 5$ ไม่ว่าง ดังนั้นลูกบอลจึงยังคงอยู่ที่ปม 2 ฟังก์ชันนี้คืนค่า `true`
5. **insert(1, 25):** ปมใบ 1 ไม่ว่าง ดังนั้นลูกบอลจึงไม่ถูกวางที่ ใบ 1 ฟัง ฟังก์ชันนี้คืนค่า `false`
6. **insert(0, 20):** ปมใบ 0 ว่าง ดังนั้นลูกบอลที่มีค่า 20 จึงถูกวางที่ปมใบ 0 และเนื่องจากปม $P[0] = 4$ ว่าง ลูกบอลจึงเคลื่อนไปยังปม 4 และเนื่องจากปม $P[4] = 6$ ไม่ว่าง ลูกบอลจึงหยุดเคลื่อนที่ ฟังก์ชันนี้คืนค่า `true`

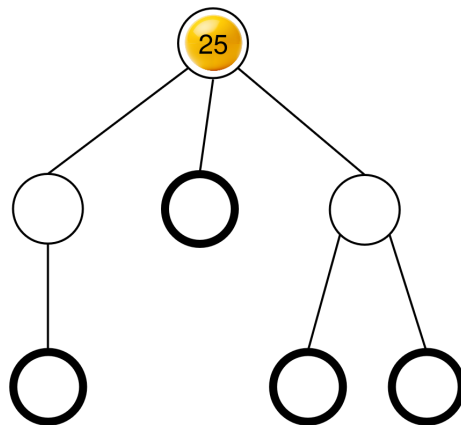
ตำแหน่งของลูกบอลทั้งหมดแสดงอยู่ในโจทย์ข้างต้นแล้ว

หลังจากนั้น ฟังก์ชันจะเรียก

```
collect()
```

การทำงานของฟังก์ชันนี้เป็นไปตามที่อธิบายในโจทย์ข้างต้น ดังนั้นฟังก์ชันนี้จึงคืนค่า $S = [0, 10, 20, 30, 20]$ หลังจากนั้น เครื่องยิงลูกบอลจะกลับมาว่างอีกครั้ง

ต่อมาฟังก์ชันสามารถเรียก `insert(2,25)` ปมใบ 2 ว่าง ดังนั้นลูกบอลที่มีค่า 25 จึงถูกวางที่ปมใบ 0 ลูกบอลนี้จะเคลื่อนที่ไปยังปม 5 แล้วไปยังปม 6 และหยุดอยู่ที่ปมนั้น ฟังก์ชันนี้คืนค่า `true` ตำแหน่งของลูกบอลแสดงดังรูปต่อไปนี้



สุดท้ายแล้วฟังก์ชันนี้สามารถเรียก `collect()` ซึ่งคืนค่า $S = [25]$ และทำให้เครื่องยิงลูกบอลว่างอีกครั้ง

สำหรับตัวอย่างนี้ มีอาร์เรย์ของปมพ่อที่ต้องอยู่สองแบบที่ `find_structure` สามารถคืนมาได้ คือ $R = P = [4, 6, 5, 5, 6, 6]$ (ซึ่งตรงกับการระบุหมายเลขปม $L = [0, 1, 2, 3, 4, 5, 6]$) และ $R = [5, 6, 4, 4, 6, 6]$ (ซึ่งตรงกับการระบุหมายเลขปม $L = [0, 1, 2, 3, 5, 4, 6]$) ในตัวอย่างนี้ $K = 2$ และ $B = 30$ ดังนั้น $C = 32$.

เกรตเตอร์ตัวอย่าง

รูปแบบข้อมูลนำเข้า:

```
N M
P[0] P[1] P[2] ... P[N-2]
```

รูปแบบข้อมูลส่งออก:

```
K B C
Q
R[0] R[1] R[2] ... R[Q-1]
```

Q คือความยาวของอาร์เรย์ R ที่คืนมาโดยฟังก์ชัน `find_structure`